Ben Moritz
benmo@pdx.edu

Planet of the Rust-Bots

Planet of the Rust-Bots is a simulation of a planet populated by self-replicating robots which aims to produce emergent behavior from the interactions between these robots and the planet itself. In order to replicate, robots will need to consume resources contained within the planet. These resources are finite, and will eventually be exhausted. Once that happens, the robots will need to find fresh deposits or acquire minerals from other robots. Over time, the robots in this simulation should change behavior, as options become limited and, eventually, the planet will be completely consumed.

GitHub Repository: https://github.com/benmo-pdx/Planet-of-the-Rust-Bots.git

The Planet

This is the universe where the simulation takes place, represented as a 6 connecting grids of identical size configured in a cube. The size of the grids is derived by the radius of the planet, which can be configured as a parameter at the beginning of the simulation. This coordinate system offers several advantages.

- Coordinate locations are distributed near-uniformly across the planet face.
- Each square on every grid is surrounded by the same number of squares, even at edges and corners.
- This planet can be rendered graphically using a Cubed Sphere method.

Each coordinate on the grid system will contain a tracked quantity of resources, which will be depleted over time. The initial deposit size will also be configurable as a parameter. Potentially, initial deposit sizes will vary over the surface of the planet, creating regions which are resource rich or poor, as this has the potential of influencing robot behavior in interesting ways.

The simulation will incorporate a helper function in order to accommodate smooth movement and searching across grid edges.

The Rust-Bots

The subjects of this simulation will be represented by data structures occupying a coordinate location on the planet called Rust-Bots. The primary function of these structures will be to generate copies of themselves in adjacent, unoccupied spaces by expending resources contained within their current grid location. They will also have the ability to carry out behaviors such as movement or combat, in addition to self-replication, which will hopefully produce interesting interactions and more complex behaviors across the simulation at large. The Rust-Bots will possess the basic behaviors listed below.

Movement: Rust-Bots can move from their current position to an adjacent space, so long as it is empty. If other actions are not possible, a Rust-Bot may default to this action, although it can still choose this action when others are available.

Replication: Rust-Bots can use resources to produce new Rust-Bots. These new bots will derive their stats from the parent bot's stats, type, and disposition. This means that the new Rust-Bot may be an exact copy, a modified copy, or a completely different bot from their parent, depending on the structure of the simulation.

Mining: Rust-Bots can transfer resources from their current location into an internal store which they will use for replication. In order for this action to be possible, some resources must be contained at their current coordinates.

Combat: When a Rust-Bot encounters another bot, it has a chance to attack it, influenced by the Rust-Bot's disposition and relation to the other bot. The outcome of combat is determined probabilistically, taking into account the stats of both Rust-Bots. This action may interrupt the action of another bot.

Idle: If no other action is possible, Rust-Bots will default to idling. A bot may still choose this action when other actions are available.

Intended action will be determined for each bot on the planet before any actions are carried out and then evaluated by grouped action in order to account for actions like combat changing a bot's intended action and to avoid conflicts, such as two bots attempting to move into the same space.

Rust-Bots will all possess a set of attributes whose values will be contained within their data fields. These attributes will include the coordinates of the current location, how far the bot can see, stored as a radius, amount of contained resources, and an ID for the bot's lineage. The simulation will contain a separate data structure formatted as a tree which tracks Rust-Bot lineage. This is to prevent bots from immediately attempting to attack their parent or child upon creation, as closely related Rust-Bots will not fight.

Several tentative fields include a maximum resource capacity, a combat power stat, an enumerated disposition variable which will influence the likelihood of different actions being taken, and an age stat which will cause the bot to break down after a certain amount of time.

The Simulation

The simulation will be initialized by generating a planet as described above according to parameters which can be set prior to running the simulation. These will govern the size of the planet, resource distribution, number of initial Rust-Bots, and initial Rust-Bot placement.

Once the planet is initialized, the simulation will evaluate "turns" until the simulation is ended or all Rust-Bots are destroyed. At each turn, initial action will be evaluated for each Rust-Bot on the planet. If any bots intend to attack, their target will be evaluated so that their action can be updated to combat. Once combat is evaluated, all remaining actions are evaluated, with target location for movement and replication being determined as the Rust-Bot takes its action, with the

possibility for that action to default to idle, so that situations where all available locations are filled prior to the action are accounted for.

Rust-Bot behavior during the simulation will be influenced by several parameters that the user can set, which are vision-radius, aggressiveness, minimum relation for combat, and general disposition.

Concerns

While this simulation could be displayed as a list or simple statistics, it would be much more interesting to render graphically in realtime. Representing the simulation as ASCII art in console is one option for this, but it would not communicate the 3D nature of the planet well. For this reason, I think it would be best to use a graphics library to achieve some basic 3D rendering. Choosing the best library, however, requires some consideration, as there are several options available, and working with the library presents some potential challenges, as I have not worked with any Rust graphics libraries before.